KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Health Report Explainer & Healthcare Information Assistant: A Retrieval-Augmented Generation (RAG) System

Project ID: PRJ053

Team ID: T053

Team GitHub Repository: xxxxxxxx

Student Name	Registration Number	Individual GitHub Link
GARLAPATI HARSHITA	99230040303	xxxxxxxx
SUNKU REDDY ANANYA SREE	99230040207	xxxxxxxx
PEDDAPPAIAH GARI KOUSHIK KUMAR	99230040398	xxxxxxxx
PONKALA BHANUPRAKASH REDDY	99230040892	xxxxxxxx

Project Trainer / Guide
Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction.....Section 1
2. Problem Statement & Objectives ..... Section 2
3. System Architecture (RAG Pipeline)..... Section 3
4. Implementation Details & Tech Stack ..... Section 4
5. Results and Testing .....Section 5
6. Conclusion & Future Scope..... Section 6

List of Figures and Tables

Figure 3.1: Overall Architecture of the Health Report Explainer ..... Section 3
Figure 3.2: Workflow of Report Analysis and Explanation ..... Section 3
Table 3.1: Components of the Architecture and Their Roles ..... Section 3
Table 4.1: Technology Stack..... Section 4
Table 4.2: FastAPI Endpoints..... Section 4
Table 5.1: Test Cases .....Section 5

1. Introduction

1.1 Background

Everyone gets a medical report at some point in life, such as a blood test, sugar test or kidney test. These reports are written for doctors. They contain medical words, short test names, numbers, units and reference ranges. A normal person looking at a report finds it difficult to know which value is normal, which value is high or low, and whether it is something to worry about.

1.2 Why This Project

Large Language Models (LLMs) such as Google Gemini can explain medical text in simple words. But an LLM answers from what it learned during training. It may miss important details or give an answer that is not supported by a reliable source. This is called hallucination. In healthcare, a wrong or incomplete explanation can confuse or even scare the user.

To solve this, we use Retrieval-Augmented Generation (RAG). In RAG, the system first searches a trusted medical knowledge base and finds the passages related to the report. These passages are then given to the LLM along with the report data, so the explanation is based on real reference material.

1.3 About Retrieval-Augmented Generation (RAG)

RAG works in two steps. First is retrieval, where the most relevant pieces of information are searched from a knowledge base using embeddings (numbers that represent the meaning of text). Second is generation, where the LLM writes the final answer using both the user's data and the retrieved information. It is like writing an exam with an open book instead of writing from memory.

1.4 About Our Project

Our project is the Health Report Explainer & Healthcare Information Assistant. The user uploads a medical report as a PDF or an image and selects a language. The system extracts the test values, compares them with the reference ranges given in the report, retrieves related medical information using RAG, and sends everything to Google Gemini. Gemini returns a simple explanation in the selected language, for example Telugu, and advises the user to consult a qualified healthcare professional.

2. Problem Statement & Objectives

2.1 Problem Statement

Understanding medical reports is difficult for ordinary people because reports contain complex medical terminology, numerical test results and clinical reference ranges. Patients may struggle to identify abnormal values and understand their possible meaning without professional help. This is more difficult for people who are not comfortable with English, because most reports are written in English.

2.2 Limitations of Existing Approaches

- Many AI chatbots explain reports using only their pre-trained knowledge, without checking any reliable medical reference.
- Such explanations can be incomplete, too general or not properly connected to the actual values in the report.
- Searching the internet for each test name takes time, and the information found may be confusing or unreliable.

2.3 Proposed Solution

We propose an AI-powered Health Report Explainer that reads the uploaded report, finds the values and reference ranges, retrieves relevant medical knowledge using RAG, and uses Google Gemini to generate a simple and multilingual explanation. Because Gemini receives retrieved medical information along with the report data, the explanation is better supported and more complete. For example, a user uploads a blood test report and selects Telugu. The application extracts the report information, retrieves medical knowledge about tests like haemoglobin, glucose and creatinine, sends both to Gemini, and displays the explanation in Telugu.

2.4 Objectives

- To build a web application that accepts medical reports in PDF and image format.
- To extract test names, values, units and reference ranges from the report using PyMuPDF and Tesseract OCR.
- To compare every value with its reference range and identify values that are low, normal or high.
- To build a medical knowledge base and store it in a vector database (ChromaDB) for semantic search.
- To retrieve the most relevant medical passages for the report using Sentence Transformers and cosine similarity.
- To use Google Gemini to generate simple explanations in the user's selected language, with advice to consult a qualified healthcare professional.

3. System Architecture (RAG Pipeline)

The architecture of our application is divided into two phases. The first phase is Building the Medical Knowledge Base, which is done once before users start using the system. The second phase is Report Analysis, which runs every time a user uploads a report. FastAPI acts as the central controller that connects all the parts.

3.1 Overall Architecture

Figure 3.1 shows the overall architecture. The user works only with the frontend. The frontend talks only to our FastAPI backend and never directly to Gemini or the vector database. This keeps the API key safe and lets the backend control the full process.

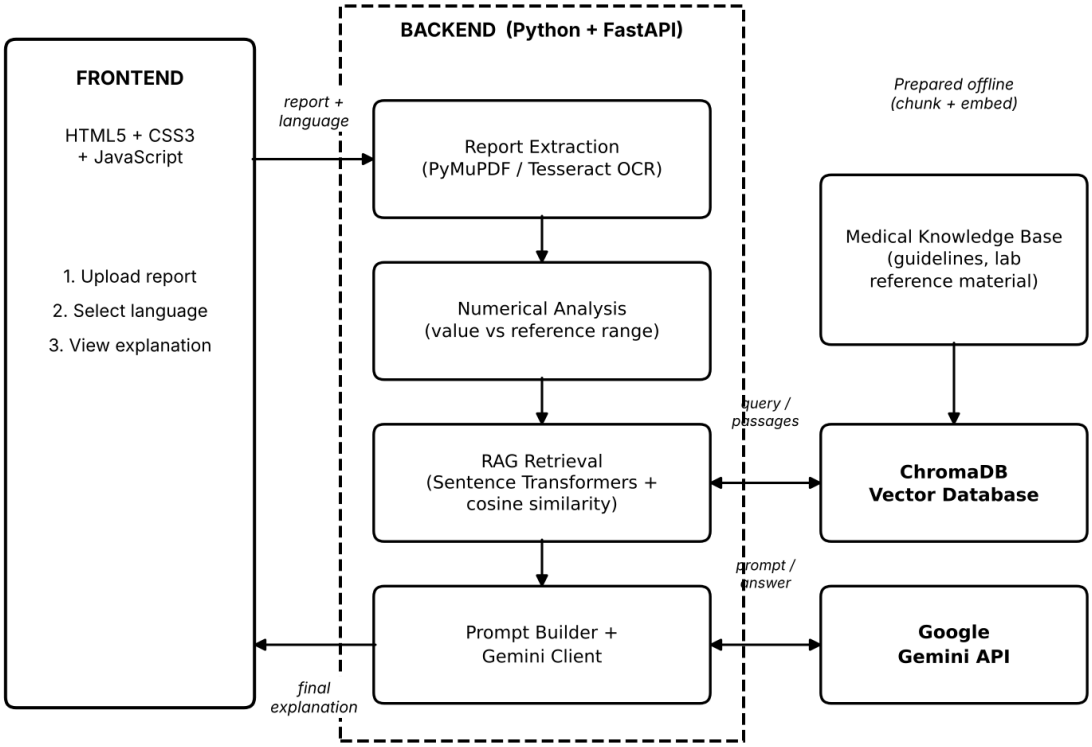


Figure 3.1: Overall Architecture of the Health Report Explainer

Table 3.1: Components of the Architecture and Their Roles

Component	Role in the System
Frontend (HTML, CSS, JavaScript)	Takes the report file and language from the user, sends them to the backend and shows the final explanation.
FastAPI Backend	Receives the request, runs extraction, analysis and retrieval, calls Gemini and returns the result.
Report Extraction	Uses PyMuPDF for PDF text and Tesseract OCR for images to get the text of the report.
Numerical Analysis	Python code that compares each value with the reference range given in the report.
RAG Retrieval	Converts the query into an embedding and finds the most similar medical passages.
ChromaDB	Vector database that stores the embeddings of the medical knowledge base.
Gemini API	Generates the final simple explanation in the selected language.

3.2 Phase A: Building the Medical Knowledge Base

This phase is done before users start uploading reports. Its aim is to prepare trusted medical knowledge that can be searched quickly.

1. **Collect trusted documents:** medical guidelines, laboratory reference material and health information documents are collected.
2. **Extract text:** PyMuPDF reads the documents and extracts the readable text.
3. **Chunking:** long text is split into small passages so that each passage covers one topic. Small chunks give more accurate search results.
4. **Embedding:** Sentence Transformers converts each chunk into a vector (a list of numbers that represents its meaning).
5. **Storage:** the vectors and their text are stored in ChromaDB, ready for similarity search.

3.3 Phase B: Workflow When a User Uploads a Report

Figure 3.2 shows the complete workflow from upload to the final explanation. The steps are explained below.

1. **Upload:** the user uploads a report and selects a language. JavaScript sends both to the FastAPI endpoint /api/analyze.
2. **File type check:** if the file is a PDF, PyMuPDF extracts the text. If it is an image or scanned report, Tesseract OCR reads the text from it.
3. **Parsing:** Python picks out the test names, values, units and reference ranges from the extracted text.
4. **Numerical analysis:** each value is compared with its reference range and marked as low, normal or high. This is done by code, not by the AI, so the comparison is exact.

5. **Retrieval (RAG):** a query is made from the report (for example the names of abnormal tests) and converted into an embedding. ChromaDB finds the most similar medical passages using cosine similarity.
6. **Generation:** Gemini receives the report information, the comparison results, the retrieved medical passages and the selected language.
7. **Explanation:** Gemini writes a simple explanation that says which values are outside the range, what they may mean, and advises the user to consult a healthcare professional.
8. **Display:** the backend returns the response and JavaScript shows it in the results area of the webpage.

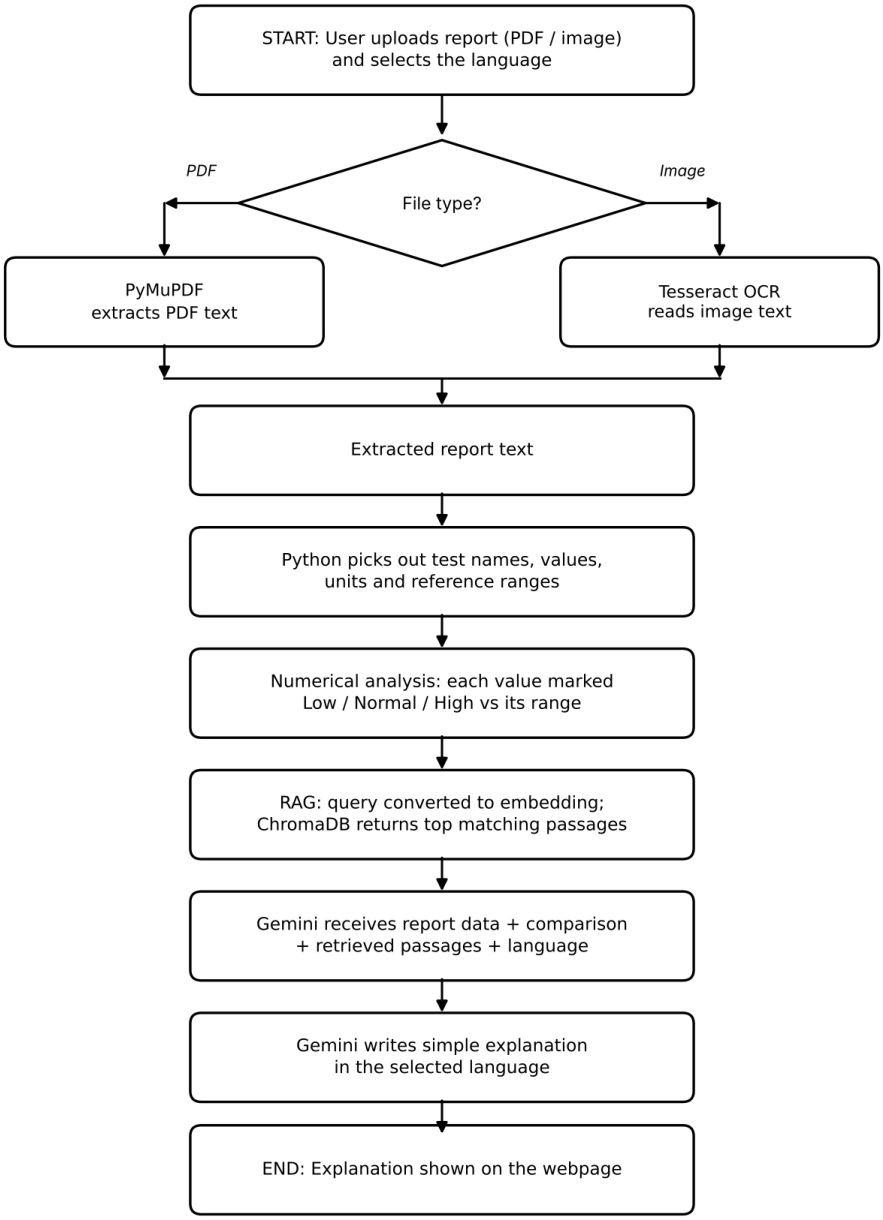


Figure 3.2: Workflow of Report Analysis and Explanation

3.4 What Gemini Receives

The prompt sent to Gemini has four parts: (1) the extracted report information, (2) the numerical comparison results, (3) the medical passages retrieved by RAG, and (4) the language chosen by the user. The prompt also tells Gemini to use simple words, to base the explanation on the given information, and to recommend consulting a healthcare professional. This is how retrieval and generation work together.

4. Implementation Details & Tech Stack

4.1 Technology Stack

Table 4.1: Technology Stack

Component	Technology Used	Purpose
Frontend	HTML5, CSS3, JavaScript	Webpage structure, styling, upload handling and API communication
Programming Language	Python	Main backend logic
Backend Framework	FastAPI	Handles requests and controls the whole processing
PDF Extraction	PyMuPDF	Extracts text from PDF reports and reference documents
Image Extraction	Tesseract OCR	Reads text from scanned reports and images
AI API	Google Gemini API	Generates explanations in the chosen language
Embeddings	Sentence Transformers	Converts medical text into vectors
Vector Database	ChromaDB	Stores and searches medical knowledge vectors
Retrieval Method	Cosine Similarity	Finds the most relevant medical passages
Configuration	.env file	Stores API keys and settings

4.2 Module Description

Frontend. Built with HTML5, CSS3 and JavaScript. HTML gives the page structure, CSS makes it neat and responsive, and JavaScript handles file upload, language selection, calling the backend and showing the result.

Extraction module. PyMuPDF extracts text from PDF files. When the report is a scanned image, or no text can be extracted normally, Tesseract OCR is used to read the text from the image.

Numerical analysis module. Python extracts test names, values, units and reference ranges and compares each value with the range given in the report. The output tells which values are low, normal or high.

RAG module. Sentence Transformers creates the embeddings. ChromaDB stores the embeddings of the medical knowledge base and searches them. Cosine similarity measures how close the query is to each stored passage, and the top matches are returned.

Gemini module. The backend prepares the prompt with the report data, comparison results, retrieved passages and language, sends it to the Gemini API and receives the explanation.

Configuration. The Gemini API key and other settings are kept in a .env file so they are not written inside the code.

4.3 FastAPI Endpoints

The frontend communicates with the backend through the following HTTP endpoints.

Table 4.2: FastAPI Endpoints

Endpoint	Purpose
/api/analyze	Uploads and analyzes the report (extraction, comparison, RAG and Gemini)
/api/upload	Existing upload functionality
/api/sample-report	Provides sample report functionality for demonstration
/api/health	Checks whether the backend is running

4.4 Safety Measures

- The frontend never calls Gemini or ChromaDB directly. Only the backend does.
- The API key is stored in the .env file and not shared with the frontend.
- The system only explains the report. It does not diagnose, and every answer advises consulting a qualified healthcare professional.

5. Results and Testing

5.1 Testing Approach

The application was tested module by module and then as a complete system. Sample blood test reports containing values such as haemoglobin, glucose and creatinine were used. The built-in sample report option (/api/sample-report) was also used to check the full flow from upload to display.

5.2 Test Cases

Table 5.1: Test Cases

No.	Test Case	Input	Expected Result
1	PDF report	Text-based blood test PDF	Text extracted by PyMuPDF; values and ranges identified
2	Image report	Scanned report image (JPG/PNG)	Text read by Tesseract OCR
3	Abnormal value	Value outside the reference range	Value marked low/high; explanation mentions its possible meaning
4	Normal values	All values within range	Marked normal; explanation says so and still advises a doctor visit
5	Language selection	Telugu selected	Explanation shown in Telugu
6	Sample report	Call to /api/sample-report	Sample report analysed and explanation displayed
7	Server check	Call to /api/health	Response confirms the backend is running

5.3 Results

- The system extracts test names, values, units and reference ranges from the report.
- Values outside the reference range are identified by Python before Gemini is called, so the comparison is exact.
- The retrieved medical passages give Gemini extra context, so the explanation is more complete than an answer from the model's own knowledge.
- The explanation is shown in the language chosen by the user, such as Telugu.
- Every explanation includes a recommendation to consult a healthcare professional.

6. Conclusion & Future Scope

6.1 Conclusion

The Health Report Explainer helps ordinary users understand their medical reports easily. By combining report extraction, numerical analysis, RAG with ChromaDB and Sentence Transformers, and Google Gemini, the system gives simple, well-supported and multilingual explanations. The project shows how RAG can make an LLM's answers more reliable by giving it trusted information to work with. The system does not replace a doctor. It only helps users understand their reports and encourages them to seek professional advice.

6.2 Limitations

- OCR may make mistakes on blurry or low-quality images.
- The quality of the explanation depends on the quality and size of the medical knowledge base.
- The system relies on the reference ranges printed in the report.
- It is an information tool and cannot give a diagnosis.

6.3 Future Scope

- Support for more languages and more types of medical reports.
- A larger knowledge base with more trusted medical sources.
- Voice output so that users can listen to the explanation.
- User login to save past reports and track values over time.
- A feedback option to keep improving the quality of explanations.